

A Practical Guide to Generative adversarial network

TOPIC -- GENERATIVE ADVERSARIAL NETWORK

$$\min_G \max_D \mathbb{E}[\log D(x)] + \mathbb{E}[\log(1 - D(G(z)))]$$

-- 01 --

In modern probability theory based on measure theory, a probability space also needs to be equipped with a σ -algebra. As a result, a more rigorous definition of the GAN game would make the following changes: Each probability space $(\Omega, \mathcal{B}, \mu)$ defines a GAN game. The generator's strategy set is $\mathcal{P}(\Omega, \mathcal{B})$, the set of all probability measures μ_G on the measure-space $(\Omega, \mathcal{B})$.

-- 02 --

(Ω_C, μ_C) , the fixed random information generator. There are 3 players in 2 teams: generator, Q , and discriminator. The generator and Q are on one team, and the discriminator on the other team. The objective function is $L(G, Q, D) = L_{GAN}(G, D) - \lambda \hat{I}(G, Q)$ where $L_{GAN}(G, D) = \mathbb{E}_{x \sim \mu_{ref}} [\ln D(x)] + \mathbb{E}_{z \sim \mu_Z} [\ln(1 - D(G(z, c)))]$ is the original GAN game objective, and $\hat{I}(G, Q) = \mathbb{E}_{z \sim \mu_Z, c \sim \mu_C} [\ln Q(c | G(z, c))]$

-- 03 --

Two time-scale update rule The two time-scale update rule (TTUR) is proposed to make GAN convergence more stable by making the learning rate of the generator lower than that of the discriminator. They prove that when trained this way, GANs "converge, under mild assumptions to a stationary local Nash equilibrium". They further show that this property extends to the use of the Adam optimizer, which is commonly used in stochastic gradient descent. It is important

to note, however, that a local Nash equilibrium in no way signifies an absence of mode collapse - for instance, a GAN trained on MNIST collapsed to generating a single digit may satisfy the hypotheses of the paper, while still presenting mode collapse.

-- 04 --

Invertible data augmentation When there is insufficient training data, the reference distribution μ_{ref} cannot be well-approximated by the empirical distribution given by the training dataset. In such cases, data augmentation can be applied, to allow training GAN on smaller datasets. Naïve data augmentation, however, brings its problems. Consider the original GAN game, slightly reformulated as follows:

$$\min_D \mathbb{E}_{x \sim \mu_{\text{ref}}} [\ln D(x)] - \mathbb{E}_{x \sim \mu_G} [\ln (1 - D(x))] \quad \min_G \mathbb{E}_{x \sim \mu_G} [\ln (1 - D(x))] - \mathbb{E}_{x \sim \mu_{\text{ref}}} [\ln D(x)]$$

Now we use data augmentation by randomly sampling semantic-preserving transforms $T: \Omega \rightarrow \Omega$ and applying them to the dataset, to obtain the reformulated GAN game:

$$\min_D \mathbb{E}_{x \sim \mu_{\text{ref}}, T \sim \mu_{\text{trans}}} [\ln D(T(x))] - \mathbb{E}_{x \sim \mu_G} [\ln (1 - D(x))] \quad \min_G \mathbb{E}_{x \sim \mu_G} [\ln (1 - D(x))] - \mathbb{E}_{x \sim \mu_{\text{ref}}, T \sim \mu_{\text{trans}}} [\ln D(T(x))]$$

This is equivalent to a GAN game with a different distribution μ_{ref}' , sampled by $T(x)$, with $x \sim \mu_{\text{ref}}, T \sim \mu_{\text{trans}}$. For example, if μ_{ref} is the distribution of images in ImageNet, and μ_{trans} samples identity-transform with probability 0.5, and horizontal-reflection with probability 0.5, then μ_{ref}' is the distribution of images in ImageNet and horizontally-reflected ImageNet, combined. The result of such training would be a generator that mimics μ_{ref}' . For example, it would generate images that look like they are randomly cropped, if the data augmentation uses random cropping. The solution is to apply data augmentation to both generated and real images:

$$\min_D \mathbb{E}_{x \sim \mu_{\text{ref}}, T \sim \mu_{\text{trans}}} [\ln D(T(x))] - \mathbb{E}_{x \sim \mu_G, T \sim \mu_{\text{trans}}} [\ln (1 - D(T(x)))] \quad \min_G \mathbb{E}_{x \sim \mu_G, T \sim \mu_{\text{trans}}} [\ln (1 - D(T(x)))] - \mathbb{E}_{x \sim \mu_{\text{ref}}, T \sim \mu_{\text{trans}}} [\ln D(T(x))]$$

$$\mathbb{E}_{\mu_{\text{ref}}, T} \left[\mathbb{E}_{\mu_{\text{trans}}} \left[\ln D(T(x)) \right] - \operatorname{operatorname} \{E\}_{\mu_{\text{sim}} \mu_{\text{G}}, T} \left[\mathbb{E}_{\mu_{\text{trans}}} \left[\ln(1-D(T(x))) \right] \right] \right] \min_{\mu_{\text{G}}} \mathbb{E}_{\mu_{\text{G}}, T} \left[\mathbb{E}_{\mu_{\text{trans}}} \left[\ln(1-D(T(x))) \right] \right] \end{cases}$$

The authors demonstrated high-quality generation using just 100-picture-large datasets. The StyleGAN-2-ADA paper points out a further point on data augmentation: it must be invertible. Continue with the example of generating ImageNet pictures. If the data augmentation is "randomly rotate the picture by 0, 90, 180, 270 degrees with equal probability", then there is no way for the generator to know which is the true orientation: Consider two generators G, G' , such that for any latent z , the generated image $G(z)$ is a 90-degree rotation of $G'(z)$. They would have exactly the same expected loss, and so neither is preferred over the other. The solution is to only use invertible data augmentation: instead of "randomly rotate the picture by 0, 90, 180, 270 degrees with equal probability", use "randomly rotate the picture by 90, 180, 270 degrees with 0.1 probability, and keep the picture as it is with 0.7 probability". This way, the generator is still rewarded to keep images oriented the same way as un-augmented ImageNet pictures. Abstractly, the effect of randomly sampling transformations $T: \Omega \rightarrow \Omega$ from the distribution μ_{trans} is to define a Markov kernel $K_{\text{trans}}: \Omega \rightarrow \mathcal{P}(\Omega)$. Then, the data-augmented GAN game pushes the generator to find some $\hat{\mu}_G \in \mathcal{P}(\Omega)$, such that $K_{\text{trans}} * \mu_{\text{ref}} = K_{\text{trans}} * \hat{\mu}_G$ where $*$ is the Markov kernel convolution. A data-augmentation method is defined to be invertible if its Markov kernel K_{trans} satisfies $K_{\text{trans}} * \mu = K_{\text{trans}} * \mu' \implies \mu = \mu' \forall \mu, \mu' \in \mathcal{P}(\Omega)$. Immediately by definition, we see that composing multiple invertible data-augmentation methods results in yet another invertible method. Also by definition, if the data-augmentation method is invertible, then using it in a GAN game does not change the optimal strategy $\hat{\mu}_G$ for the generator, which is still μ_{ref} . There are two prototypical examples of invertible Markov kernels: Discrete case: Invertible stochastic matrices, when Ω is finite. For example, if $\Omega = \{\uparrow, \downarrow, \leftarrow, \rightarrow\}$ is the set of four images of an arrow, pointing in 4 directions, and the data augmentation is "randomly rotate the picture by 90, 180, 270 degrees with

probability p , and keep the picture as it is with probability $(1 - 3p)$, then the Markov kernel K_{trans} can be represented as a stochastic matrix: $[K_{\text{trans}}] = \begin{bmatrix} (1 - 3p) & p & p & p \\ p & (1 - 3p) & p & p \\ p & p & (1 - 3p) & p \\ p & p & p & (1 - 3p) \end{bmatrix}$ and K_{trans} is an invertible kernel iff $[K_{\text{trans}}]$ is an invertible matrix, that is, $p \neq 1/4$. Continuous case: The gaussian kernel, when $\Omega = \mathbb{R}^n$ for some $n \geq 1$. For example, if $\Omega = \mathbb{R}^{256 \times 256}$ is the space of 256×256 images, and the data-augmentation method is "generate a gaussian noise $z \sim N(0, I_{256 \times 256})$, then add ϵz to the image", then K_{trans} is just convolution by the density function of $N(0, \epsilon^2 I_{256 \times 256})$. This is invertible, because convolution by a gaussian is just convolution by the heat kernel, so given any $\mu \in \mathcal{P}(\mathbb{R}^n)$, the convolved distribution $K_{\text{trans}} * \mu$ can be obtained by heating up $\mathbb{R}^n$ precisely according to μ , then wait for time $\epsilon^2/4$. With that, we can recover μ by running the heat equation backwards in time for $\epsilon^2/4$. More examples of invertible data augmentations are found in the paper.